

DOCKER MASTERY

A Story of Learning — From Zero to Expert

Every concept explained through real questions and real confusion

Built from an actual learning conversation — the questions are real, the struggle is real

6

chapters

60+

code blocks

35+

Q&A moments

12

core topics

How This Book Was Born

This book is different from every Docker tutorial you have read. It was not written by an expert sitting down to explain Docker to beginners. It was born from a real learning conversation — question by question, confusion by confusion, breakthrough by breakthrough. Every chapter follows the same arc you experienced when learning: the question that exposed a gap in understanding is shown in purple, the explanation follows, then the code that makes it concrete, then the deeper question that came next.

That flow is preserved intentionally. Learning works best when you see not just the answer but the exact question that unlocked it. When you read 'why specifically this path `/var/lib/apt/lists/*?`' — that was a real question. When you see 'so if I inject a secret at runtime will it appear in docker history?' — that was a real moment of someone thinking precisely enough to ask the right follow-up. Those questions are the map of someone genuinely understanding a system, not just memorising it.

Why Story-Flow Learning Works

Reading a textbook is like being handed a finished map. This book is like watching the map being drawn — you see where the confusion was, where the wrong turn happened, and why the correct path makes sense. That is a fundamentally different kind of understanding — one that sticks.

By the end you will not just know Docker commands. You will know the thinking behind them, the questions that reveal when to use them, and the mental models that make the whole system feel inevitable rather than arbitrary.

Contents

Chapter 1 — How Docker Actually Works Inside

- › The container is just a Linux process
- › Linux namespaces — 6 walls of isolation
- › cgroups — the resource governor
- › The union filesystem and copy-on-write
- › Hands-on proof at the kernel level
- › Containers vs virtual machines

Chapter 2 — The Dockerfile — Every Instruction

- › Layer caching — the rule that changes everything
- › Multi-stage builds — compile here, ship there
- › Anti-pattern 1: apt-get without cleanup
- › Why /var/lib/apt/lists/* specifically
- › Anti-pattern 2: hardcoded secrets
- › Anti-pattern 3: the :latest tag
- › ENTRYPOINT vs CMD — the deepest difference
- › ENV vs ARG — build time vs runtime
- › Does runtime injection appear in history?
- › WORKDIR — why cd does not work
- › USER — never run as root
- › EXPOSE and what it actually does
- › The --chown flag on COPY

Chapter 3 — The .dockerignore File

- › Build context — what gets sent to Docker
- › Every pattern rule with examples
- › The * vs ** trap everyone hits
- › Rule order — last match always wins
- › What to include and exactly why
- › Security category: non-negotiables
- › Size category: biggest practical impact
- › Correctness category: the sneaky one
- › The inverted whitelist pattern

Chapter 4 — Volumes and Data Persistence

- › Why containers lose data — the proof
- › Named volumes
- › Bind mounts — no copying needed
- › The dev workflow with auto-reload
- › tmpfs mounts
- › When to use each type

Chapter 5 — Docker Networking

- › The apartment building model
- › Default bridge — why it fails
- › Custom bridge — DNS by container name
- › Port publishing — every pattern decoded
- › docker run vs docker start vs docker exec
- › Network isolation for security

Chapter 6 — Real Commands Explained

- › docker run -it — every flag
- › The .whl install command fully decoded
- › Layer size analysis for Jetson Dockerfile
- › The ip addr not found mystery
- › Build context size check
- › docker system df — understanding disk usage

How Docker Actually Works Inside

Namespaces, cgroups, and the truth about containers

Before any command makes sense, you need to understand what Docker actually *is* at the kernel level. Most tutorials skip this entirely — which is why most people feel like they are memorising magic spells instead of understanding a system. Once you see what is really happening, every Docker behaviour stops being surprising and starts being obvious.

1.1 The Container Is Just a Process

The most important thing to understand about Docker: a container is **not** a virtual machine. It is not a separate operating system. It is not a simulation of another computer. A **container is a regular Linux process** running on your computer — but with two kernel features wrapping it to create the illusion of isolation.

- **Namespaces** — create the illusion that the process is alone in its own world
- **cgroups (control groups)** — limit how much CPU, RAM, and disk that process can consume

That is the entire secret. No hypervisor, no hardware emulation, no separate kernel. Docker is a tool that sets up namespaces and cgroups, then starts a process inside them. The process itself is completely real and visible from the host machine.

The Restaurant Kitchen

Imagine you are running a restaurant. Normally every chef works in the same shared kitchen. If one chef burns something, the smoke fills the entire room. If one chef uses all the olive oil, everyone else has none. Containers solve this by giving each chef their own private kitchen booth — same building, same gas supply, completely isolated space. That is exactly what Linux namespaces do for processes.

1.2 Linux Namespaces — The 6 Walls of Isolation

Linux has 6 types of namespaces. Docker uses all of them together. Each one creates a different dimension of isolation:

Namespace	What it isolates	What the container sees
pid	Process list	Only its own processes. bash thinks it is PID 1.
net	Network stack	Its own eth0, its own IP address, its own iptables rules.
mnt	Filesystem	Its own / — completely unaware of the host's real filesystem.
uts	Hostname	Its own hostname — shows the container ID, not your machine name.
ipc	Inter-process communication	Shared memory, semaphores — isolated from all other processes.
user	User and group IDs	Container's root (uid 0) maps to a non-root user on the host.

The pid namespace is worth understanding specifically. When you run **ps aux** inside a container you only see bash and ps — the container's isolated process view. From the host machine, that same bash process has a completely different PID number (like 12345) and is visible alongside hundreds of other host processes. Same process, completely different view depending on which namespace you are looking from.

1.3 cgroups — The Resource Governor

While namespaces create isolation, cgroups enforce resource limits. Without cgroups, one misbehaving container could eat all your RAM and crash the entire machine. cgroups are how Docker gives you the **--memory** and **--cpus** flags.

cgroups under the hood

```
# When you run this:
docker run --memory=512m --cpus=0.5 myapp

# Docker creates kernel entries at:
# /sys/fs/cgroup/memory/docker/<id>/memory.limit_in_bytes
# /sys/fs/cgroup/cpu/docker/<id>/cpu.cfs_quota_us

# The Linux kernel enforces these at the hardware level
# App tries to use more than 512 MB → kernel kills it

# Check if a container was killed by OOM:
docker inspect myapp --format '{{.State.OOMKilled}}'
# true = was killed by out-of-memory
```

1.4 The Union Filesystem — Why Layers Exist

Transparent Slides

Think of a Docker image like a stack of transparent slides on an overhead projector. Each slide adds something. The bottom slide is the base OS. The next adds Python. The next adds your packages. The last adds your code. The projector shows all slides combined — that is what the container sees as its filesystem. Crucially, if two images share the same bottom slides, Docker stores those slides only once on disk.

Docker uses a storage driver called **overlay2** on modern Linux. It has three parts:

- **Lower dir (read-only)** — all the image layers stacked. Immutable — nothing can change them.
- **Upper dir (read-write)** — a thin writable layer unique to each running container. This is where file changes go.
- **Merged view** — what the container sees: lower and upper combined into one seamless filesystem.

When a container modifies a file from a lower layer, overlay2 copies that file up to the upper dir first — then modifies it there. The original in the lower dir is completely untouched. This is called **copy-on-write (CoW)**. The image never changes no matter what the container does.

1.5 Hands-On Proof — The Container Is a Process

Do not just read this. Run these commands. Seeing the container process in the host's process table makes Docker stop feeling like magic.

Inside the container — the isolated view

```
# Terminal 1 — start a container
docker run -it --name demo ubuntu bash

# INSIDE the container — check your process list
ps aux
# You only see bash and ps
# The container's isolated process view via pid namespace

# Check your network
ip addr
# You see eth0 with 172.17.x.x — NOT your Mac's network
# The container's own network stack via net namespace

# Check your hostname
hostname
# Shows the container ID like a3f2c1b9d4e5
# Not your machine name — via uts namespace
```

From the host — the truth behind the magic

```
# Terminal 2 — on your HOST machine

# Find the container's PID on the host
docker inspect demo --format '{{.State.Pid}}'
# Returns a number like 12345

# That bash process IS a normal host process
ps aux | grep 12345
# You see it running — just a regular Linux process

# See the 6 namespace files for that process
ls -la /proc/12345/ns
# Shows: ipc mnt net pid user uts
# These are the 6 isolation walls Docker created
```

KEY INSIGHT

Container is process 12345 wearing a costume made of namespaces. Inside, bash thinks it is PID 1 on a machine called 'a3f2c1b9d4e5' with IP 172.17.0.2. From the host it is just process 12345. That is the entire secret of Docker. Everything else follows from this one fact.

YOUR QUESTION

I ran `docker inspect myapp --format '{{json .GraphDriver}}'` and it returns null. The container is running though.

That is completely normal Mac behaviour — not an error. Docker Desktop on Mac runs all containers inside a hidden Linux VM. The GraphDriver information — the overlay2 paths — lives inside that VM's filesystem.

When you inspect from your Mac terminal, Docker returns null for GraphDriver because those paths are inaccessible from outside the VM.

Mac vs Linux — What You See Is Different

Everything you are learning is correct and working exactly as described. The examples in this book are written for Linux because that is what all production servers run. On Mac, Docker Desktop adds a VM layer between you and the kernel that hides the low-level details. When you work on a real Linux server or a cloud VM, GraphDriver will show the full overlay2 LowerDir, UpperDir, MergedDir paths exactly as described.

1.6 Containers vs Virtual Machines — The Real Difference

Aspect	Virtual Machine	Container
What it runs	A full separate OS kernel	A process on the HOST kernel
Boot time	30 to 120 seconds	Milliseconds
Size	Gigabytes — full OS	Megabytes — just app and libs
Isolation	Hardware-level via hypervisor	Kernel-level via namespaces
Overhead	High — dedicated CPU and RAM	Near zero — shared kernel
Security boundary	Strong — separate kernel	Weaker — shared kernel
Best use	Different OS, maximum isolation	Same OS, fast and lightweight

Security Consideration

Containers share the host kernel. This means a kernel exploit could potentially escape container isolation. For workloads requiring strong security boundaries — financial systems, multi-tenant user code execution — consider VMs or tools like gVisor that add a layer between the container and the kernel.

The Dockerfile — Every Instruction

Building images the professional way

You have seen Dockerfiles before. Now we go deeper — not just what each instruction does, but why it exists, the subtle traps that catch even experienced engineers, and the mental models that make the right choice obvious.

2.1 Layer Caching — The Rule That Changes Everything

Docker executes Dockerfile instructions top to bottom. When a layer changes, all layers below it are invalidated and must be rebuilt. This is not a limitation — it is a feature. Understanding it lets you design Dockerfiles where a code change only rebuilds the last layer, saving minutes on every build.

The Photo Roll

Think of each layer as a photograph taken at that moment in time. If you change something in photo 3 of 10, photos 4 through 10 need to be retaken. But photos 1 and 2 are fine — they are already developed. Your goal is to put things in the right sequence so that the most frequently changing things are always near the end of the roll.

	What goes here	Frequency of change
	FROM base image	Almost never — rarely needs rebuild
	RUN apt-get install system packages	Rarely — only when you add/remove a dep
	COPY requirements.txt + RUN pip install	Only when dependencies change
	COPY . . (your application code)	Every single commit — but only this layer rebuilds

Layer ordering — bad vs good

```
# BAD — app code near the top poisons everything below it
FROM python:3.11-slim
COPY . /app                # changes every commit
RUN apt-get install gcc    # REBUILD on every code change — 45 sec wasted
RUN pip install -r requirements.txt # REBUILD too — 60 sec wasted

# GOOD — app code at the bottom where it belongs
FROM python:3.11-slim
RUN apt-get install gcc    # CACHED — runs once
COPY requirements.txt .    # CACHED — only runs if deps change
RUN pip install -r requirements.txt # CACHED — only runs if deps change
COPY . /app                # REBUILDS on code change — fast, expected
```

The pip install trick is worth examining closely. By copying **only** requirements.txt before running pip install, you ensure that changing any .py file in your project does not invalidate the pip cache. The pip layer is only rebuilt when requirements.txt actually changes. This alone can save 60 to 120 seconds on every single build in a typical ML project.

2.2 Multi-Stage Builds — Compile Here, Ship There

YOUR QUESTION

Explain the multi-stage build Dockerfile — what is happening in each stage?

Multi-stage builds solve a specific problem: building ML extensions requires gcc, cmake, and BLAS headers. These build tools add 800 MB to your image but are completely useless at runtime. The solution is to compile in stage 1 (which gets discarded), then copy only the compiled artifacts to stage 2 (which ships to production).

Construction Site vs Finished Building

Stage 1 is the construction site — it has all the heavy tools: cranes, concrete mixers, scaffolding, generators. Once the building is finished you do not ship the construction site to the client. Stage 2 is the finished building — only what people need to actually live and work in it. Docker discards the construction site automatically.

The Three Most Common Multi-Stage Patterns

	Stage 2 (runtime)	Size reduction
+ compile .so files	python:slim + runtime libs + compiled .so	1.8 GB →
go build	scratch (empty) + single static binary	870 MB →
es + webpack build	node:alpine + prod deps only + /dist folder	1.1 GB →

Complete multi-stage ML Dockerfile

```
# Stage 1 — the builder (discarded after build completes)
FROM python:3.11 AS builder
RUN apt-get install -y gcc cmake libopenblas-dev
# ↑ heavy tools needed to compile C extensions

COPY requirements.txt .
RUN pip install numpy scipy --no-binary :all:
# --no-binary forces compilation from source
# needs gcc and BLAS headers — only possible in stage 1

COPY . /app
RUN python setup.py build_ext --inplace
# produces .so compiled extension files

# Stage 2 — the runtime (this is what ships to production)
FROM python:3.11-slim
# ↑ completely fresh start — zero memory of stage 1

RUN apt-get install -y libopenblas-runtime
# only the runtime library, not the 380 MB dev headers

COPY --from=builder /app/build/*.so /app/
# ↑ the bridge — copy only compiled artifacts from builder
# gcc, cmake, BLAS headers never enter this image
```

COPY --from=builder — The Bridge Between Stages

The `--from=builder` flag is the key instruction. It tells Docker: reach into the builder stage's filesystem (which still exists in the build cache at this point) and copy these specific files into the current stage. Everything that existed in the builder but was not explicitly copied simply ceases to exist in the final image. The 1.5 GB toolchain never boards the ship.

The word **builder** is not special — it is just the name you gave the stage with **AS builder**. You can name it anything. You can also reference stages by index (`--from=0`) but names are more readable.

2.3 Anti-Pattern 1 — apt-get Without Cleanup

YOUR QUESTION

Why specifically delete `/var/lib/apt/lists/*` and is that path important?

When you run `apt-get update`, apt goes out to every repository in your sources list and downloads their **index files — the full catalogue of every package** available. For a standard Ubuntu image this is 90 to 120 MB of plain-text index files. Once apt-get install has used these to find and download your packages, they serve zero purpose forever after.

The path `/var/lib/apt/lists/` is specifically where apt stores these catalogue files. The largest one — **universe_binary-amd64_Packages** — is 62 MB by itself because it contains every community-maintained package. Your container downloads this even if you never install a single universe package, because the universe repository is enabled by default.

apt-get — the only correct pattern

```
# WRONG — cleanup in a separate RUN
RUN apt-get update
RUN apt-get install -y gcc cmake
RUN rm -rf /var/lib/apt/lists/*
# Layer 1 sealed with 90 MB of package lists
# Layer 3 runs rm but only MASKS layer 1 – bytes still there
# Docker layers are immutable snapshots

# RIGHT — everything in one RUN
RUN apt-get update && \
    apt-get install -y --no-install-recommends gcc cmake && \
    apt-get clean && \
    rm -rf /var/lib/apt/lists/*
# Cleanup happens BEFORE Docker seals the layer
# The 90 MB is already gone when Docker takes the snapshot
# Saves 90–120 MB every single build
```

The `apt-get clean` handles the downloaded `.deb` package files at `/var/cache/apt/archives/`. The `rm -rf /var/lib/apt/lists/*` handles the index catalogues. You need both. And the `--no-install-recommends` flag prevents apt from installing all the 'recommended' packages (man pages, GUI tools, locale data) — just install what you explicitly asked for. This combination typically saves 100 to 150 MB on a typical ML image.

Why the /* Matters

The * glob at the end matters. `rm -rf /var/lib/apt/lists/*` deletes the contents of the directory but keeps the directory itself. apt expects the directory to exist. If you accidentally delete the directory itself (forgetting the /*), future apt-get calls fail with 'no such file'. The directory stub is zero bytes so there is no reason to remove it.

2.4 Anti-Pattern 2 — Secrets in the Dockerfile

Never hardcode secrets — API keys, passwords, tokens — in a Dockerfile. Every ENV and ARG instruction is recorded in the image's metadata and visible to anyone with access to the image.

Secrets in Dockerfile — what gets exposed

```
# NEVER DO THIS
ENV DB_PASSWORD=s3cr3t123
ENV API_KEY=sk-abc123xyz

# Anyone can run:
docker history myimage
# ENV DB_PASSWORD=s3cr3t123 ← exposed permanently

# Or inspect any container from this image:
docker inspect <container> | grep Env
# 'DB_PASSWORD=s3cr3t123' ← also visible

# Unsetting in a later layer does NOT help
ENV DB_PASSWORD=
# The original value is still in a previous layer's snapshot
```

	Visible in docker inspect?	Safe for s
	YES	NEVER
	Sometimes	NEVER
	YES (container only)	Acceptable
	NO	Best for bu
	NO	Best for ru

YOUR QUESTION

So if I inject a secret at runtime with `docker run -e`, it won't appear in history?

Correct. Runtime injection never touches the image at all — so it cannot appear in history. The key is understanding what docker history actually records.

Runtime injection — clean image history

```
# docker history records events during docker BUILD
# docker run -e happens AFTER the build is finished
# The image is already sealed — history is already written
```

2.5 Anti-Pattern 3 — The :latest Tag

The :latest tag is not a guarantee of anything. It is simply a pointer that image maintainers move when they release a new version. When the Python team releases 3.13, they update the :latest pointer. Your CI pipeline pulls it, your build suddenly has a different Python that breaks your C extensions, changes SSL behaviour, or removes a deprecated API — with no change in your Dockerfile.

Pinning — wrong vs right

```
# WRONG — :latest is a moving target
FROM python:latest
# Today: Python 3.12.3
# Next month: Python 3.13.0 — might break your extensions

# WRONG — minor tag can get patch updates
FROM python:3.11
# Gets silent patch updates you did not ask for

# RIGHT — pin to exact patch version and variant
FROM python:3.11.9-slim
# This specific image never changes
# Upgrades are a conscious, reviewable code change

# MAXIMUM — digest pin (immune to tag rewrites)
FROM python@sha256:ab12cd34ef56...
# Survives even if someone force-pushes to the same tag
```

Automate the Upgrade

Pinning is not a reason to never update. Use Dependabot or Renovate to automatically open pull requests when a new patch is released. You get reproducibility AND security patches — without surprise breakage in the middle of a sprint.

2.6 ENTRYPOINT vs CMD

YOUR QUESTION

Explain ENTRYPOINT and CMD — what is the difference and when do I use each?

Job Title and Default Task

Think of ENTRYPOINT as the job title and CMD as the default task. If you hire someone as a 'Python runner' (ENTRYPOINT), they always run Python — that is their identity. But their default script might be 'main.py' (CMD). You can tell them to run a different script today without changing their job title. If you just use CMD with no ENTRYPOINT, you are hiring a general worker — tell them nothing and they do their default, tell them something and they do that instead.

What it controls

What happens when you type after image name

The entire default command	CMD is completely replaced by whatever you type
The fixed executable — the container's identity	Your text is APPENDED as arguments to ENTRYPOINT
ENTRYPOINT fixed, CMD = default arguments	CMD is replaced, ENTRYPOINT stays — best of both

Three ENTRYPOINT/CMD patterns

```
# Pattern 1: CMD alone — maximum flexibility
FROM python:3.11-slim
CMD ["python", "main.py"]

# docker run myimage          → python main.py
# docker run myimage bash      → bash    (CMD thrown away completely)
# docker run myimage python -c '1' → python -c '1'

# Pattern 2: ENTRYPOINT alone — container IS python
FROM python:3.11-slim
ENTRYPOINT ["python"]

# docker run myimage          → python (interactive shell)
# docker run myimage main.py  → python main.py
# docker run myimage bash     → python bash ← error! python runs bash as script
# docker run --entrypoint bash myimage → bash ← only way to override ENTRYPOINT

# Pattern 3: ENTRYPOINT + CMD — best for production services
FROM python:3.11-slim
ENTRYPOINT ["python", "serve.py"]
CMD ["--port", "8080", "--workers", "4"]

# docker run myimage          → python serve.py --port 8080 --workers 4
# docker run myimage --port 9090 → python serve.py --port 9090
# CMD replaced, ENTRYPOINT stays — override defaults without changing the command
```

Why Always Use Exec Form — The JSON Array

You will notice all examples above use the JSON array format: `["python", "main.py"]`. Never use the shell form: `python main.py`. The difference is critical in production.

	What actually runs	Problem
	<code>/bin/sh -c 'python main.py'</code> — sh is PID 1	Your app never receives SIGTERM
	<code>python main.py</code> directly — python is PID 1	Receives SIGTERM cleanly

When Docker stops a container it sends SIGTERM. If `/bin/sh` is PID 1 (shell form), sh usually does not forward signals to child processes. Your app never knows it should shut down. Docker waits 10 seconds then force-kills it. Database connections are not closed, in-flight requests are dropped, files may be partially written. Exec form makes your process PID 1 directly — it receives SIGTERM, handles it, and shuts down cleanly.

2.7 ENV vs ARG

YOUR QUESTION

What is the difference between ENV and ARG?

These two look similar but live in completely different moments of Docker's lifecycle:

Visible in	Override with
docker history + docker inspect + ps aux	docker run -e VAR=value
docker history (mostly)	docker build --build-arg VAR=value

ENV vs ARG in practice

```
# ARG — only during docker build
ARG PYTHON_VERSION=3.11
FROM python:${PYTHON_VERSION}-slim
# ARG is gone once image is built
# Not available inside running container

# ENV — survives into the running container
ENV APP_PORT=8080
ENV MODEL_PATH=/models
ENV LOG_LEVEL=INFO

# Inside the container your app reads these normally:
# import os
# port = os.environ['APP_PORT']    # '8080'
# level = os.environ['LOG_LEVEL']  # 'INFO'

# Override at runtime — no rebuild needed:
# docker run -e LOG_LEVEL=DEBUG -e APP_PORT=9090 myimage
```

ENV also works inside the Dockerfile itself after it is set. You can use **\$VAR_NAME** syntax in subsequent instructions:

ENV used inside Dockerfile

```
ENV APP_HOME=/app
ENV MODEL_PATH=/models

WORKDIR $APP_HOME      # creates and sets /app as working dir
RUN mkdir -p $MODEL_PATH # creates /models directory
COPY . $APP_HOME       # copies into /app

# Change APP_HOME in one place → everything below updates
```

2.8 WORKDIR — Never Use cd

YOUR QUESTION

What exactly does WORKDIR do and why can't I just use cd in a RUN instruction?

The answer lies in understanding what RUN actually does. When Docker processes a RUN instruction it spawns a brand new /bin/sh process, runs your command inside it, captures the filesystem changes as a new layer, then **kills the shell process completely**. The shell's memory of where it was — its current directory — dies with it.

The Call Centre

Every RUN instruction is like calling a customer support agent. You tell this agent 'go to folder /app'. They do it. Call ends — agent gone. New agent picks up for the next RUN. They have zero memory of the previous call. They start at the root /. WORKDIR is like leaving a standing instruction at the call centre: all agents must start at /app. Now every agent automatically starts there without you having to tell them.

WORKDIR vs cd

```
# WRONG — cd does not survive between RUN instructions
RUN cd /app                                # shell 1: moves to /app
                                           # shell 1 dies — /app is forgotten
RUN pip install -r requirements.txt         # shell 2: starts at /
                                           # FAILS — no requirements.txt at /

# Also WRONG — cd in same RUN works but COPY is still at /
RUN cd /app && pip install -r requirements.txt
COPY . . # still copies to / because WORKDIR was never set

# RIGHT — WORKDIR persists across ALL subsequent instructions
WORKDIR /app

COPY requirements.txt . # . means /app — correct
RUN pip install -r requirements.txt # runs from /app — correct
COPY . . # copies into /app — correct
CMD ["python", "main.py"] # looks for main.py in /app — correct
```

WORKDIR also creates the directory automatically if it does not exist — unlike cd which would fail with 'no such file'. And it affects interactive sessions: when you run **docker exec -it mycontainer bash** you land directly in /app rather than having to cd every time.

YOUR QUESTION

So docker run -it image bash takes us inside the image filesystem like a Linux machine, but WORKDIR /app creates an app folder and stores everything there, and without WORKDIR files go in the root folder?

Yes, exactly right. Let me confirm that precisely:

WORKDIR — the practical difference you see

```
# WITHOUT WORKDIR — files land at /
FROM python:3.11-slim
COPY . .

# docker run -it myimage bash
# root@container:/# ls
# bin boot dev etc home lib usr var ...
# app.py main.py requirements.txt ← mixed with Linux system files
```

KEY INSIGHT

WORKDIR does not only affect where files go. It affects every instruction after it: COPY . . lands in /app, RUN pip install runs from /app and finds requirements.txt, CMD python main.py looks for main.py in /app, docker exec bash lands in /app. One setting — consistent everywhere.

2.9 USER — Never Run as Root

By default, containers run as root (uid 0). This feels convenient but is dangerous. If your app has a vulnerability and an attacker gets code execution, running as root means they can read any file in the container, modify system files, install tools, and potentially find a path to escape to the host machine. Running as a non-root user dramatically limits what a successful attacker can do.

Secure Dockerfile — non-root user

```
FROM python:3.11-slim

# Step 1: Create a group (like a department)
# --gid 1001 ← explicit group ID, avoids conflicts with system groups
RUN groupadd --gid 1001 appgroup

# Step 2: Create the user (like a new employee)
# --uid 1001 ← explicit user ID
# --gid appgroup ← put user in this group
# --shell /bin/bash ← give user a shell
# --create-home ← create /home/appuser directory
# appuser ← the username
RUN useradd --uid 1001 --gid appgroup --shell /bin/bash \
    --create-home appuser

WORKDIR /app

# Install deps as root – needs write access to system dirs
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

# Copy files AND set ownership in one step
# Without --chown, files are owned by root
# appuser would get 'permission denied' writing to them
COPY --chown=appuser:appgroup . .

# Switch to non-root for the application process
USER appuser
# Everything after this runs as uid 1001, not root

CMD ["python", "main.py"]
```

Why 1001 Specifically?

By Linux convention, UIDs **0 to 999 are reserved** for system accounts. Regular user accounts start at 1000. Using 1001 signals that this is an application user, not a system account. The order in the Dockerfile matters:

you install packages as root first because pip needs to write to system directories like `/usr/local/lib/python3.11/site-packages/`, then you switch to appuser. The switch is permanent for that stage — once USER appuser is set, everything after runs as that user.

2.10 EXPOSE — Documentation Not Magic

EXPOSE does not publish a port. It does not open any holes. It does not make the container accessible from outside. It is purely documentation — **it tells other developers and Docker Compose which port your app listens on inside the container.** The actual port opening only happens with the **-p** flag in docker run.

What EXPOSE actually does

```
EXPOSE 8080
# Means: 'my app listens on 8080'
# Does NOT make port 8080 accessible from outside

# To actually publish it:
# docker run -p 8080:8080 myimage

# EXPOSE is useful because:
# 1. docker run -P (capital P) publishes all EXPOSEd ports randomly
# 2. Docker Compose uses EXPOSE to know which port to map
# 3. Documentation for anyone reading your Dockerfile
# 4. Container orchestrators use it for service discovery
```

CHAPTER 3

The .dockerignore File

What to exclude, why, and every pattern rule explained

When you run **docker build .**, the dot at the end means 'current directory'. **Docker takes your entire current directory, packages it all up, and sends it to the Docker daemon before a single line of the Dockerfile runs. That transfer is called the build context.** A missing or incomplete .dockerignore file is one of the most common causes of unnecessarily slow builds.

Check your build context size — do this first

```
# Check what your build context size actually is
docker build --no-cache . 2>&1 | head -3

# Good — .dockerignore working
# Sending build context to Docker daemon 4.2MB

# Bad — .dockerignore incomplete
# Sending build context to Docker daemon 2.3GB
# 2.3 GB transferred BEFORE a single instruction runs
# This happens on EVERY docker build — every commit in CI

# 2>&1 — redirects stderr to stdout
# (Docker prints this line to stderr, not stdout)
# head -3 — show only first 3 lines
```

The target build context size for a typical ML project should be under 50 MB. If it is larger, you have a missing .dockerignore problem. On a team where 10 people run **docker build** 20 times per day, a 2 GB context wastes enormous time every single day.

3.1 Every Pattern Rule Explained

The * Wildcard — Single Level Only

The star matches any sequence of characters within a **single path segment**. The critical detail: it does not cross directory boundaries. This is the single most common .dockerignore mistake.

* does not cross directory boundaries

```
# *.pyc only matches at the ROOT DIRECTORY LEVEL
*.pyc
# MATCHED:      app.pyc
# NOT MATCHED:  src/app.pyc
# NOT MATCHED:  a/b/c/app.pyc

# This is why your __pycache__ files might still appear
# even though you think *.pyc is in your .dockerignore
```

The ** Double Wildcard — Any Depth

Double star matches across directory boundaries — any number of folders deep. This is what you almost always need for Python artifacts.

**** crosses directory boundaries**

```
# **/*.pyc matches at ANY DEPTH
**/*.pyc
# MATCHED: app.pyc
# MATCHED: src/app.pyc
# MATCHED: a/b/c/app.pyc

# For ML projects — catch everything everywhere
**/__pycache__/*
**/*.pyc
**/*.pyo
```

The / Trailing Slash — Directories Only

Without a trailing slash, a pattern matches both files and directories with that name. The trailing slash makes it unambiguous — directories only.

Trailing slash — directory specificity

```
# Without slash — matches file OR directory named 'models'
models

# With slash — matches ONLY the models DIRECTORY
models/

# Practical difference:
# models      → would also match a file called 'models'
# models/     → would NOT match a file called 'models.py'

# Always use trailing slash when you mean a directory:
.venv/
__pycache__/*
data/
models/
.git/
```

The ! Negation — Re-include Exceptions

A pattern starting with ! re-includes a file that was previously excluded by an earlier pattern. It means: 'wait, except this one'. The key rule: the ! must come **after** the excluding pattern.

! negation — order matters critically

```
# Correct order — exclude first, then re-include
*.log          # exclude all log files
!important.log # wait — include this specific one back

# error.log      → matched by *.log → IGNORED
# access.log     → matched by *.log → IGNORED
```

The ? Single Character — Exact One Match

? single character

```
# ? matches exactly ONE character
app?.py
# MATCHED:      app1.py  appA.py  appX.py
# NOT MATCHED: app.py   (0 characters)
# NOT MATCHED: app12.py (2 characters)

# Useful for versioned files:
config?.yaml
# matches config1.yaml, config2.yaml but not config.yaml
```

Comments — Document Your Decisions

Comments — use them liberally

```
# Lines starting with # are ignored by Docker

# Python artifacts
**/__pycache__/*
**/*.pyc

# ML model weights — these are mounted at runtime via volume
# See docker-compose.yml volumes section
models/
*.pt
*.pth
```

3.2 Rule Order — Last Match Always Wins

This is the most misunderstood part of `.dockerignore`. Docker reads every pattern from top to bottom. For each file, the last matching pattern determines whether it is included or excluded. Later rules override earlier ones.

	Result	Why
	IGNORED	*.log match
	INCLUDED	*.log match
	IGNORED	! runs first
	IGNORED	docs/ match
	INCLUDED	docs/ match

3.3 What to Include — Organized by Why

Category 1: Security — Non-Negotiable

These files contain credentials. If they get into an image pushed to any registry — even a private one — every person with pull access can read your secrets by running `docker inspect` or simply `cat`-ing the file inside a container.

Security exclusions — never skip these

```
# Secrets — the non-negotiables
.env
*.env
*.key
*.pem
*.p12
secrets/
credentials/
.aws/
.gcloud/
.kube/
.ssh/

# Note: once a secret is in an image that was pushed to a registry,
# you cannot remove it. You must rotate the secret immediately.
# There is no undo for a pushed image.
```

Category 2: Image Size — Biggest Practical Impact

YOUR QUESTION

Should I put `.pt` files in `.dockerignore`?

Yes, for the exact same reason as the dataset folder. `.pt` files are PyTorch model weights. They are data, not code. Your docker build process has zero use for them — they do not get installed, they do not run during the build, they just add gigabytes to your build context and get baked into the image. Mount them at runtime via a volume instead.

Size exclusions — the biggest practical wins

```
# Python virtual environments
# Built for your host OS architecture — wrong inside container AND huge
.venv/
venv/
env/

# ML model weights — mounted at runtime via volume
models/
*.pt
*.pth
*.bin
*.onnx
*.h5
*.pkl
*.safetensors

# Training data — belongs on a volume or object storage
data/
```

The mental test for any file: **does my running application need this file to serve requests?** If the answer is no — it goes in `.dockerignore`. Datasets do not serve requests. Model weights do (but they arrive via volume mount at runtime, not at build time). Log files do not. Your `.venv` does not — the container installs its own.

Category 3: Correctness — The Sneaky One

These files do not cause security issues or size problems, but they can cause subtle bugs where your build passes locally but behaves differently in CI or production.

Correctness exclusions — prevent silent bugs

```
# Local environment files with WRONG values for production
.env.local
.env.development
# These files contain:
# DB_URL=postgresql://localhost:5432/devdb
# DEBUG=true
# If they leak into the image, your production container
# connects to localhost (which does not exist) or runs in debug mode

# Test artifacts – reflect your host test run, not container
.pytest_cache/
.coverage
coverage/
htmlcov/
.tox/

# Log files from local runs – stale data, confuses log monitoring
logs/
*.log
```

Category 4: Noise — Clutter Without Consequence

Noise exclusions — clean images

```
# macOS and Windows filesystem metadata
.DS_Store
Thumbs.db
desktop.ini

# IDE settings – personal to your machine
.vscode/
.idea/
*.swp
*.swo

# CI/CD configuration – container does not need to know how it was built
.github/
.gitlab-ci.yml
.circleci/
Jenkinsfile

# Documentation – belongs in git, not in the container
```

The Inverted Whitelist Pattern

YOUR QUESTION

*What does this .dockerignore file mean? It starts with ** then uses ! to add things back.*

That is the inverted or whitelist approach to .dockerignore. The normal approach says: include everything, except these specific things. The inverted approach says: exclude everything first, then explicitly add back only what I allow.

Inverted pattern — whitelist approach

```
# Inverted .dockerignore — exclude everything first
**
# ↑ exclude literally every single file and folder

# Now explicitly allow back what you need
!Dockerfile
!docker-compose.yml
!*.py           # Python files at root level only
!*.yaml         # YAML files at root level only
!scripts/       # re-include the scripts directory
!scripts/**     # ALSO needed: re-include directory CONTENTS
# Both lines needed — !scripts/ brings back the dir,
# !scripts/** brings back what is inside it
```

The Whitelist Trap

You cannot re-include a file if its parent directory was excluded by a directory pattern. The ** exclusion excludes scripts/ as a directory. !scripts/ re-includes the directory itself but not its contents. You need !scripts/** separately. This is the trap that makes the whitelist pattern fragile. Use it only when your project structure is very messy — the standard blacklist is simpler for most projects.

Volumes and Data Persistence

Where data lives, why it disappears, and how to keep it

Whiteboard vs Notebook

A container's internal filesystem is like a whiteboard. Every time you erase and redraw — stop and recreate the container — you start with a blank board. Volumes are like a notebook that sits next to the whiteboard. You write important things in the notebook, and it survives no matter how many times the whiteboard gets erased. The whiteboard is the container's writable layer. The notebook is the volume.

4.1 Why Containers Lose Data — The Proof

This catches almost every Docker beginner. The container's writable layer — the thin read/write layer sitting on top of the read-only image layers — is **tied to the container's lifetime**. When you delete a container with `docker rm`, that writable layer is deleted with it. The image survives untouched, but every change your app made at runtime is gone.

Prove data loss — do this exercise

```
# PROVE IT — run this sequence yourself

# Step 1: Start a container and create a file
docker run -it --name test ubuntu bash
echo 'important data' > /myfile.txt
exit

# Step 2: Container is stopped but still EXISTS — file is still there
docker start test
docker exec test cat /myfile.txt
# Output: important data ← survived stop and restart
# Key: docker stop does NOT delete the writable layer

# Step 3: DELETE the container
docker rm test
# The writable layer is now deleted from disk

# Step 4: New container from the same image
docker run -it --name test2 ubuntu bash
cat /myfile.txt
# cat: /myfile.txt: No such file or directory
# THE FILE IS GONE — the image never had it
# test2 has a brand new empty writable layer
```

Container state	Writable layer / data
Paused — container still exists on disk	SURVIVES — layer preserved

Running again	INTACT — same layer resumed exactly
Permanently deleted	GONE FOREVER — layer removed with container
New container from image	Brand new empty writable layer — clean slate

4.2 Named Volumes — Docker Manages the Storage

A named volume is storage that Docker manages on your host machine at `/var/lib/docker/volumes/`. It exists completely independently of any container. You refer to it by name, and Docker handles the rest. The key property: **docker rm does not touch named volumes**.

Named volume — data survives container deletion

```
# Create a named volume
docker volume create pgdata

# Mount it — container writes to /var/lib/postgresql/data
# which is actually the pgdata volume on your host
docker run -d \
  --name postgres \
  -e POSTGRES_PASSWORD=secret \
  -v pgdata:/var/lib/postgresql/data \
  postgres:16

# Insert some data
docker exec -it postgres psql -U postgres -c \
  "CREATE TABLE users (id serial, name text);"
docker exec -it postgres psql -U postgres -c \
  "INSERT INTO users (name) VALUES ('arobin');"

# DELETE the container entirely
docker rm -f postgres

# Create a completely new container — same volume
docker run -d --name postgres2 \
  -e POSTGRES_PASSWORD=secret \
  -v pgdata:/var/lib/postgresql/data \
  postgres:16

# Query the new container
docker exec -it postgres2 psql -U postgres -c \
  "SELECT * FROM users;"
# id | name
# ----+-----
#  1  | arobin ← data survived container deletion
```

4.3 Bind Mounts — Your Files Inside the Container

A bind mount maps a specific directory from your host machine directly into the container. Changes on either side are **immediately visible on both sides simultaneously**. This is not a sync — it is a direct link. The same bytes on disk, seen from two different places.

YOUR QUESTION

So if I mount a folder in Docker, do I need to copy the files into the Docker folder to run them? Let's say my local test folder has main.py — do I need to copy it from test to /app?

No. You do not need to copy anything. That is the entire point of a bind mount — the folder IS already there. No copying, no moving, nothing extra.

Photocopy vs Glass Wall

COPY in Dockerfile makes a snapshot — like photocopying a document. The copy is separate from the original. Bind mount -v creates a window directly into your folder — like a glass wall. You see the real thing, not a copy. Touch the glass from inside, the real thing moves.

Bind mount — no copying needed

```
# Your Mac: /Users/arobin/test/main.py

docker run -it \
  -v /Users/arobin/test:/app \
  -w /app \
  python:3.11-slim \
  python main.py

# Does main.py need to be copied into /app first?
# NO — /Users/arobin/test and /app are the SAME folder
# Same bytes. Same files. Docker made them one.
# python main.py finds it instantly and runs it

# Syntax breakdown:
# -v HOST_PATH:CONTAINER_PATH
# -w /app sets working directory (equivalent to WORKDIR)
# python:3.11-slim use official Python image
# python main.py override CMD — run this instead
```

The Full Development Workflow

Full dev workflow with bind mount

```
docker run -d \
  --name dev \
  -v $(pwd):/app \
  -v $(pwd)/logs:/app/logs \
  -p 8080:8080 \
  myapp python -m uvicorn main:app --reload --host 0.0.0.0

# $(pwd) — your current directory on Mac
# /app — where it appears inside the container
# They are the same folder — direct link, no copying

# Second volume: logs/ on Mac = /app/logs in container
# App writes log files to /app/logs
# You can read them in your Mac terminal instantly
```

KEY INSIGHT

The image contains your dependencies (changes rarely — rebuild rarely). The bind mount contains your code (changes constantly — no rebuild needed). Separate the two concerns and your inner development loop becomes instant.

4.4 tmpfs — RAM Storage, Gone on Restart

A tmpfs mount stores data in the host's RAM. It is fast, but disappears the moment the container stops. Use it for sensitive temporary data that must never touch disk.

tmpfs mount

```
docker run -d \
  --tmpfs /tmp:size=512m,mode=1777 \
  --tmpfs /run/secrets:size=10m \
  myapp

# size=512m    - max 512 MB of RAM for this mount
# mode=1777    - file permissions (same as /tmp convention)

# Use cases:
# /tmp - fast temporary computation, never needs to survive restart
# /run/secrets - tokens injected at runtime, never written to disk
# Sensitive ML intermediate results that must not persist
```

4.5 Volume Decision Guide

	Use	Why
mysql)	Named volume	Docker manages it, survives container delete, portable
m	Named volume	Write once, reference by name, easy to back up
ment	Bind mount	Real-time sync with your editor — no rebuild
	Bind mount	Map your data directory directly — no copy needed
ly	Bind mount	Edit on host, container sees change immediately
ta	tmpfs	Never touches disk, automatically cleaned on stop

Docker Networking

How containers find each other and talk to the world

Docker networking trips up almost everyone at first because it involves three separate concepts that are easy to confuse: how containers are isolated from each other, how containers communicate with each other, and how traffic from outside reaches containers. Each has a different mechanism.

The Apartment Building

Your computer is an apartment building. Each container is one apartment with its own locked door. By default apartments cannot communicate — they do not even know each other exists. A Docker network is the shared hallway. Once two containers are on the same hallway they can knock on each other's door using the container name as the apartment number. The `-p` flag is the building's front entrance — it lets visitors from outside reach a specific apartment.

5.1 The Default Bridge — Why It Fails

When Docker installs, it creates a network called **bridge** automatically. All containers without a specified network go here. The problem: containers on the default bridge **cannot find each other by name**. There is no DNS. You have to use IP addresses, which change on every restart.

Default bridge — no DNS

```
# Start two containers on the DEFAULT network
docker run -d --name web nginx
docker run -d --name db postgres -e POSTGRES_PASSWORD=pass

# Try to reach 'db' by NAME from 'web'
docker exec web ping db
# ping: db: Name or service not known
# FAILS — default bridge has no DNS server

# You would have to use the IP directly
docker inspect db --format '{{.NetworkSettings.IPAddress}}'
# Returns: 172.17.0.3

# This works — but it is fragile
docker exec web ping 172.17.0.3
# 172.17.0.3 changes every time db restarts
# Hardcoding IPs in connection strings breaks silently
```

5.2 Custom Bridge — DNS by Container Name

A custom bridge network has Docker's **embedded DNS server** at 127.0.0.11 inside every container. Every container on the network is automatically registered by its container name. This is what every real application uses. This is also what Docker Compose creates automatically.

Custom bridge — DNS by name

```
# Step 1: Create a custom network
docker network create myapp-net
# Docker creates a virtual switch with a new subnet
# Starts the embedded DNS server for this network

# Step 2: Start containers on that network
docker run -d --name db \
  --network myapp-net \
  -e POSTGRES_PASSWORD=secret \
  postgres:16
# Docker registers 'db' → 192.168.48.2 in the DNS server

docker run -d --name web --network myapp-net nginx
# Docker registers 'web' → 192.168.48.3 in the DNS server

# Step 3: DNS works automatically
docker exec web ping db
# PING db (192.168.48.2): 56 data bytes — IT WORKS

# Verify DNS inside the container
docker exec web cat /etc/resolv.conf
# nameserver 127.0.0.11 ← Docker's embedded DNS

docker exec web nslookup db
# Name: db Address: 192.168.48.2
```

The moment db restarts and gets a new IP address, Docker's DNS server updates automatically. Your connection string **host='db'** continues to work. The container name is stable even when the IP changes. This is why custom bridges are always the right choice for multi-container applications.

In your own Python application inside the web container:

Application connection string using container name

```
import psycopg2

# Uses container NAME as hostname — Docker DNS resolves it
conn = psycopg2.connect(
    host='db',          # container name, not an IP
    port=5432,
    database='myapp',
    password='secret'
)
# Works on any machine, any IP change, any restart
# No hardcoded IPs anywhere in your application code
```

5.3 Port Publishing — Every Pattern Decoded

YOUR QUESTION

Can you explain the -p flag and the different patterns?

A container is hidden inside your computer. Nothing outside Docker can reach it. Port publishing creates an iptables rule on your host that forwards traffic from a host port to a container port. When you type `-p 8080:80`, Docker creates a rule saying: any traffic arriving at host port 8080, forward it to this container's port 80.

	Example	What it does
	<code>-p 8080:80</code>	Host port 8080 → container port 80. Accessible from
	<code>-p 80:80</code>	Same port — simplest. Use when the port is not taken
USER	<code>-p 127.0.0.1:8080:80</code>	Localhost only — other machines on your network can
	<code>-p 80</code>	Docker picks a random host port. Use docker port to
	<code>-p 5353:5353/udp</code>	UDP traffic instead of TCP.

Multiple ports and verification

```
# Multiple ports — add a -p flag for each one
docker run -d \
  -p 80:80 \
  -p 443:443 \
  -p 9090:9090 \
  --name myapp myimage

# Port 80    → HTTP web traffic
# Port 443   → HTTPS encrypted web traffic
# Port 9090  → Prometheus metrics endpoint (convention)

# See current port mappings
docker port myapp
# 80/tcp    -> 0.0.0.0:80
# 443/tcp   -> 0.0.0.0:443
# 9090/tcp  -> 0.0.0.0:9090

# 0.0.0.0 means accessible from anywhere
# To restrict to localhost only:
# -p 127.0.0.1:9090:9090 ← metrics not exposed externally

# Verify with curl
curl http://localhost:8080
```

5.4 docker run vs docker start vs docker exec

YOUR QUESTION

What is the difference between `docker run` and `docker start`? And is `docker exec` for entering the container's bash?

These three commands look similar but do completely different things. Understanding each one is critical because using the wrong one either creates unwanted containers or fails silently.

	Creates anything?	When con
--	-------------------	----------

	YES — brand new container	Container
)	NO — resumes existing one	N/A — you
	NO — adds a process to it	Container

Three Commands — One Analogy

docker run = build a new house and move in. docker start = unlock and re-enter a house you already own. docker exec = visit someone already living in their house without disturbing them.

Correct lifecycle — run, start, exec

```
# CORRECT lifecycle for a database container

# First time — use docker run
docker run -d --name mydb postgres:16

# Stop at end of day
docker stop mydb

# Next morning — use docker start, NOT docker run
docker start mydb
# data is still there, same container resumed

# Enter the running container to inspect
docker exec -it mydb bash
# -i = interactive (keep stdin open so you can type)
# -t = allocate a terminal (so it feels like a real shell)
exit
# Container keeps running — you just left the visit

# Common mistake: using docker run again
docker run -d --name mydb postgres:16
# Error: container name 'mydb' already exists
# OR worse — creates 'mydb2', data seems to be lost
```

The `-it` flags on `exec` deserve explanation. `-i` keeps stdin open so you can type commands. `-t` allocates a pseudo-terminal so the shell feels like a real interactive session with proper formatting and colour. Without `-t` you get raw output without line editing. Without `-i` you cannot type. You almost always want both together when entering a shell.

5.5 Network Isolation — Security by Design

Two custom networks on the same host are completely isolated from each other. A container on network A cannot reach a container on network B. This lets you design secure multi-tier architectures.

Network isolation — frontend cannot reach database

```
# Create two isolated networks
docker network create frontend-net
docker network create backend-net
```


Real Commands Explained

Decoding every flag from actual commands you encountered

6.1 The .whl Install Pattern — Fully Decoded

YOUR QUESTION

Explain this command: `pip install --no-cache-dir --no-deps --ignore-requires-python /tmp/torchvision-0.20.0a0+afc54f7-cp310-cp310-linux_aarch64.whl && rm /tmp/torchvision-0.20.0a0+afc54f7-cp310-cp310-linux_aarch64.whl`

The command

```
RUN pip install --no-cache-dir --no-deps --ignore-requires-python \
/tmp/torchvision-0.20.0a0+afc54f7-cp310-cp310-linux_aarch64.whl && \
rm /tmp/torchvision-0.20.0a0+afc54f7-cp310-cp310-linux_aarch64.whl
```

Flag by Flag

--no-cache-dir — Do not save the downloaded wheel to pip's local cache directory after installing. Normally pip caches packages so reinstalling is faster. Inside a Docker container you never reinstall, so the cache is pure wasted space. This flag can save 50 to 200 MB depending on your packages.

--no-deps — Install ONLY this wheel. Do not install any of its declared dependencies. This is critical here because torch and the other dependencies of torchvision were already installed in earlier layers. If pip installed them again it might pull a different version and break the carefully pinned numpy version. By saying --no-deps you are taking responsibility for having already satisfied the dependencies yourself.

--ignore-requires-python — The .whl file contains metadata saying 'this package requires Python X.Y'. If the version in the container does not exactly match the requirement in the wheel's metadata, pip refuses to install. This flag says: skip that check and install anyway. Used here because this is a custom-built wheel for Jetson hardware and the metadata may not precisely match the container's Python version.

/tmp/torchvision-0.20.0a0+afc54f7-cp310-cp310-linux_aarch64.whl — Installing from a local file rather than downloading from PyPI. The wheel was already copied into /tmp in an earlier COPY instruction.

Decoding the Wheel Filename

Part of filename	What it means
torchvision	Package name
0.20.0a0	Version 0.20.0 alpha 0 — pre-release
+afc54f7	Built from git commit afc54f7 — not an official PyPI release
cp310	Built for CPython version 3.10
cp310 (second)	Compatible with CPython 3.10
linux	Linux operating system

aarch64	ARM 64-bit architecture — that is why it is a custom wheel
---------	--

The **aarch64** part is the entire reason this wheel exists. Official PyPI only has x86_64 (Intel/AMD) wheels for torchvision. On ARM machines — your Jetson Orin Nano, your Mac M4, AWS Graviton instances — you either build from source yourself or use a pre-built ARM wheel like this one from NVIDIA.

&& rm /tmp/torchvision...whl — Delete the wheel file in the same RUN instruction after installing. The wheel is only needed during installation. Once pip has extracted and installed the package, the .whl file is dead weight. Deleting it in the same RUN instruction means it is gone before Docker seals the layer. If you deleted it in a separate RUN, the wheel would be sealed in the previous layer taking up space permanently — exactly the same principle as `rm -rf /var/lib/apt/lists/*`.

6.2 Layer Size Analysis — Your Jetson Dockerfile

YOUR QUESTION

For my Dockerfile, calculate what container size each layer will be.

Layer	Contents	Estimated size
FROM l4t-jetpack:r36.4.0	CUDA 12.6 + cuDNN + TensorRT + Ubuntu ARM64 — the unavoidable Jetson baseline	~6.1 GB
RUN apt-get install...	gcc, cmake, libav, libopenblas, ffmpeg, libgl — C runtime deps + cleanup	~380 MB
RUN wget cuSPARSElt	Sparse matrix .so library for torch 2.5 on JetPack 6	~120 MB
RUN python3.10 -m venv	Virtual environment scaffold + pip upgrade	~35 MB
numpy==1.26.4	Pinned first — must be <2.0 for compiled extensions	~35 MB
torch 2.5 wheel	CUDA kernels + C++ runtime + Python bindings — largest single layer	~1.4 GB
torchvision wheel	ARM64 custom wheel — compiled C extensions, wheel deleted after	~28 MB
Core + web stack (layer 4)	FastAPI + pydantic + supabase + polars + pillow + ~40 other packages	~285 MB
scipy + matplotlib + opencv	Data/ML stack — split because needs numpy correct first	~195 MB
ultralitics	Depends on torch + numpy + opencv all being stable	~55 MB
numpy force-reinstall	Defensive pin — ultralytics may upgrade numpy to 2.x	~35 MB
COPY codes/backend/	Your application source code	~45 MB
TOTAL estimated	Full built image	~8.7 GB

The 6.1 GB base image is unavoidable for Jetson. That is the price of GPU access on ARM hardware — the entire NVIDIA software stack (CUDA, cuDNN, TensorRT, drivers) must ship with the image. On a standard

x86 server you would start from python:3.11-slim at 130 MB. On Jetson you start at 6.1 GB. Everything the Dockerfile does above that is reasonable given what it installs.

YOUR QUESTION

docker images shows size 6.9 GB and virtual size 19.9 GB — what is the difference?

Number	Meaning
6.9 GB (SIZE)	Actual unique disk space this image occupies on your machine — what you really care about
19.9 GB (VIRTUAL)	Total of all layers including ones shared with other images — counts shared layers multiple times
~13 GB difference	Build cache — intermediate layers Docker kept from the build process

Diagnose and clean disk usage

```
# Check real disk usage by category
docker system df
# TYPE          TOTAL   ACTIVE   SIZE      RECLAIMABLE
# Images        4       2        19.9GB    13.2GB
# Build Cache   47      0        6.8GB     6.8GB (100%)
# Containers    3       1        1.2MB     800KB

# The 13 GB gap is build cache — safe to delete
# Clean only build cache (never touches images or volumes)
docker builder prune

# See exact size of each layer
docker history myimage --no-trunc

# Confirm after cleanup
docker system df
```

6.3 The ip addr Not Found Error

YOUR QUESTION

I ran ip addr inside the ubuntu container and got: bash: ip: command not found

Completely normal. The ubuntu image on Docker Hub is intentionally minimal — it contains only the absolute bare essentials to keep the image size small. Network tools like ip, ping, curl, wget are not included because most containers never need them. A running nginx container or Python API has no use for network debugging tools.

Fix: ip command not found

```
# Fix — install the tools you need
apt-get update && apt-get install -y \
    iproute2 \
    iputils-ping \
    curl \
```

This is exactly why almost every Dockerfile starts with **RUN apt-get install**. The base images are stripped down deliberately — you add only what your specific application needs. A web server does not need `iproute2`. An ML training job does not need `curl`. Start minimal, add only what you actually need. This philosophy is also why the `ubuntu` container does not have `ip` by default — most containers running in production never need to debug their own network.

6.4 Understanding docker run Flags

Let us decode a typical development docker run command flag by flag:

The command to decode

```
docker run -d \
  --name dev \
  -v $(pwd):/app \
  -v $(pwd)/logs:/app/logs \
  -p 8080:8080 \
  myapp python -m uvicorn main:app --reload --host 0.0.0.0
```

Flag / part	What it does	What happens without it
-d	Detached — run in background, return terminal immediately	Terminal stays attached to container output
--name dev	Name the container 'dev' for easy reference	Docker assigns a random name like 'vibrant_tesla'
-v \$(pwd):/app	Bind mount current directory to /app inside container	Code changes require docker build to take effect
-v \$(pwd)/logs:/app/logs	Second bind mount — log files appear on your Mac directly	Logs only accessible inside container via docker exec
-p 8080:8080	Publish host port 8080 to container port 8080	Container unreachable from browser or curl
myapp	Image to use — your custom built image with dependencies	N/A
python -m uvicorn...	Overrides CMD in Dockerfile with this specific command	Default CMD from Dockerfile runs instead
--reload	uvicorn watches for file changes and auto-restarts	Code changes require container restart to take effect
--host 0.0.0.0	uvicorn listens on all interfaces, not just localhost	Port published but unreachable — traffic cannot reach uvicorn

Where You Go From Here

You have now completed the same journey that happened in this conversation — from 'what is a container' all the way to understanding namespaces at the kernel level, every Dockerfile instruction with the reasoning behind each one, .dockerignore patterns and why specific paths matter, volumes and the proof that data disappears without them, networking from the apartment analogy to actual DNS resolution, and real production commands decoded flag by flag. That is not beginner knowledge. That is intermediate-to-advanced Docker engineering.

The pattern that made this learning work is worth naming: every concept was encountered through a real question, and every real question led to a deeper real question. 'Why this path specifically?' led to understanding the apt package index. 'Will it appear in history?' led to understanding build time vs runtime. 'Do I need to copy the files?' led to understanding what a bind mount actually is. Questions are the unit of learning.

Natural next topic	Why it follows directly	First practical step
Docker Compose	Define all containers, networks, volumes in one YAML — the real-world tool for everything in this book	Write a compose.yml for nginx + fastapi + postgres
Health checks	How Docker proves your app is actually working, not just running — critical for production	Add HEALTHCHECK to your Jetson Dockerfile
Kubernetes	Orchestrate Docker containers across many servers — the platform all MLOps tools are built on	kubectl + kind for local K8s + first Deployment
CI/CD pipelines	Automatically build, scan, test, and push your image on every git commit	GitHub Actions with docker/build-push-action
Container security	Non-root users (already done), read-only filesystems, seccomp profiles, image scanning	trivy scan + docker run --read-only
Registry and tagging	How images travel from your machine to production servers	docker push to Docker Hub or AWS ECR

KEY INSIGHT

The single rule that separates engineers who truly know Docker from those who have read about it: every concept in this book only becomes real when you type the command yourself, hit an error, figure out why, and fix it. You asked the right questions in this conversation. Keep asking them at the terminal.